



INGENIERÍA EN SISTEMAS COMPUTACIONALES

Sistemas Programables

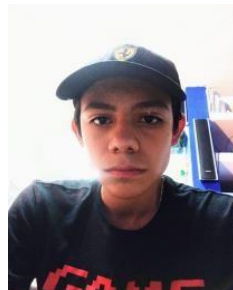
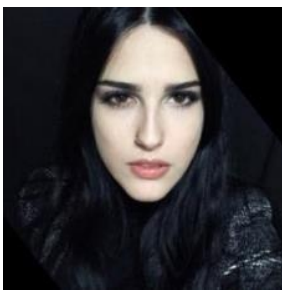
Módulo 4: Comunicaciones

Persistencia en la Nube y Dashboard de Control (Firebase ↔ Python ↔ Interfaz)

Agente Robótico Autónomo "GARRA-OS"

P R E S E N T A:

Alcalá Ramos Luz Estefanía
Bahena Mora Emilio Salvador
Casas Bastidas José Iván
Fischer González Patrick



DOCENTE:

Ma. Verónica Tapia Ibarra

1. Objetivo del proyecto

Extender el sistema GARRA-OS desarrollado en las unidades anteriores con una capa de persistencia en la nube basada en Firebase Realtime Database, junto con un dashboard web que permita supervisar el estado del robot en tiempo real y ejecutar acciones remotas sobre sus actuadores. El sistema debe respetar la privacidad del usuario, registrar eventos con marca de tiempo del servidor y mantener una arquitectura desacoplada por capas (HAL ↔ MQTT ↔ Firebase ↔ Front-end).

2. Arquitectura del sistema

La arquitectura mantiene la comunicación MQTT diseñada en la Unidad 2 (ESP32 ↔ broker público Mosquitto) y agrega un servidor puente en Python que cumple dos roles simultáneos: suscriptor MQTT que reescribe la telemetría en Firebase, y escucha activa (mediante stream) del nodo /comandos para retransmitir órdenes hacia la ESP32. El dashboard web nunca habla MQTT: solo lee y escribe en Firebase, lo que simplifica la seguridad y permite acceso desde cualquier red.

2.1 Diagrama de flujo

ESP32 (sensores y actuadores) ⇌ MQTT (Mosquitto) ⇌ Python (puente) ⇌ Firebase Realtime DB ⇌ Dashboard Web (HTML + JS + SDK modular v10).

2.2 Tópicos MQTT y rutas de Firebase

Tópico MQTT	Ruta en Firebase
garraos/01/sensor/distancia	/telemetria/distancia_cm + /eventos
garraos/01/sensor/aceleracion	/telemetria/aceleracion + /eventos
garraos/01/sensor/golpe	/alertas (severidad alta)
garraos/01/sensor/bateria	/telemetria/bateria_pct + /eventos
garraos/01/estado	/estado/online
garraos/01/cmd/servo	← /comandos/servo (dashboard)
garraos/01/cmd/oled	← /comandos/oled (dashboard)
garraos/01/cmd/buzzer	← /comandos/buzzer (dashboard)
garraos/01/cmd/paro	← /comandos/paro (dashboard)

3. Eventos almacenados en Firebase

La rúbrica exige al menos tres tipos de eventos distintos. El servidor Python escribe cada registro en `/eventos/{push_id}` con un timestamp generado por el servidor de Firebase (no por el cliente) usando el valor especial `{"sv": "timestamp"}`. Esto garantiza consistencia incluso si el reloj de la ESP32 o de la PC están desincronizados.

- **Telemetría:** lecturas periódicas de los sensores (HC-SR04, MPU6050, batería). Se actualiza la rama `/telemetry` con la última lectura y además se añade un log en `/eventos`.
- **Alertas de IA:** se generan cuando se cruza un umbral. Por ejemplo, distancia menor a 10 cm, batería por debajo de 20% o detección de un golpe en el chasis por parte del MPU6050. Se duplican en `/alertas`, donde se mantienen únicamente las últimas 5 (limpieza automática hecha por el servidor).
- **Cambios de estado de actuadores:** cada vez que el dashboard envía un comando (servo, OLED, buzzer, paro) se registra qué actuador cambió y a qué valor, lo que permite auditar quién accionó qué sobre el robot.

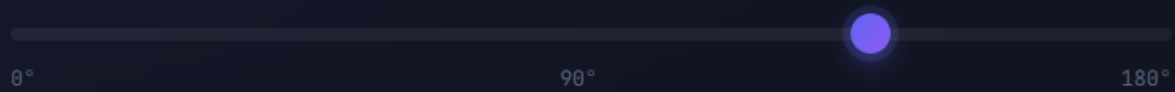


Control de actuadores

Envía órdenes en tiempo real a la ESP32

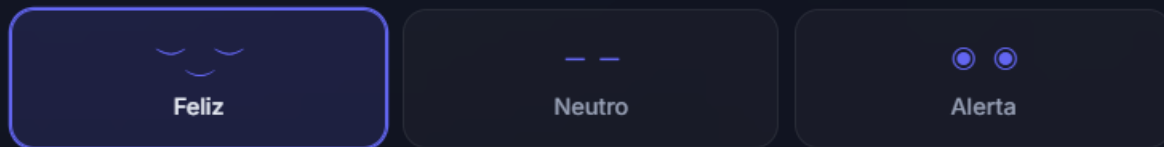
Servo del cuello

135°



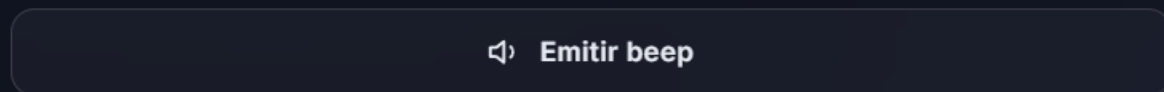
Emoción en OLED

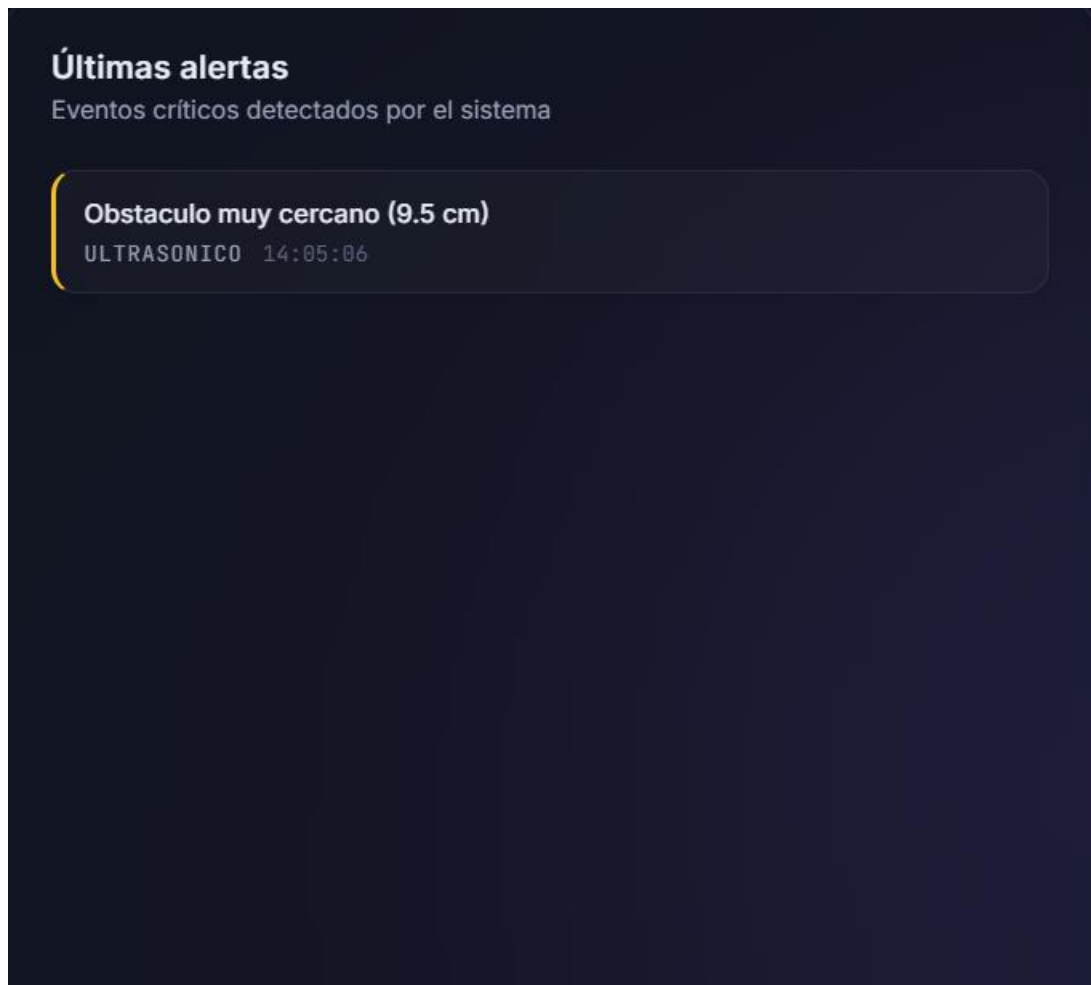
Cara mostrada en pantalla



Buzzer

Beep corto · 1000 Hz





4. Interfaz de usuario

El dashboard se desarrolló como una aplicación web estática (HTML + CSS + JavaScript modular) usando el SDK v10 de Firebase. Se eligió este enfoque sobre Flutter para evitar la curva de aprendizaje del framework y poder enfocar el tiempo en cumplir los requisitos funcionales con un diseño visual cuidado.

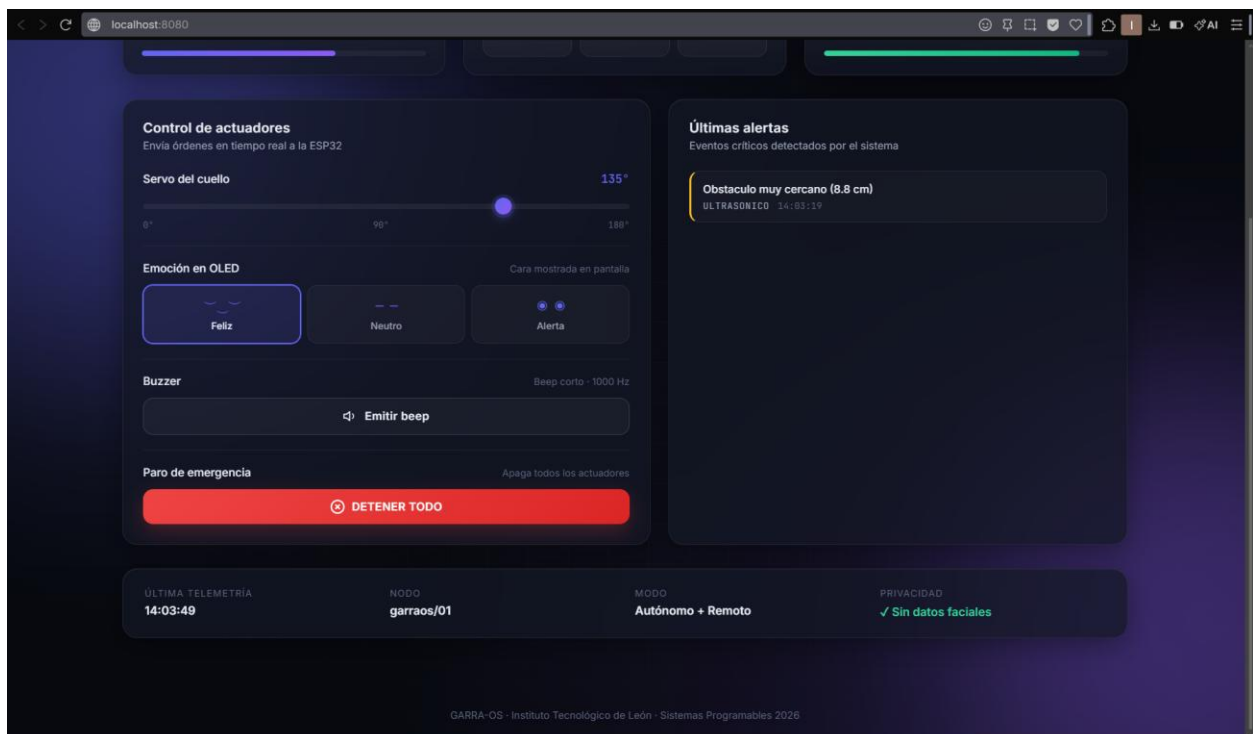
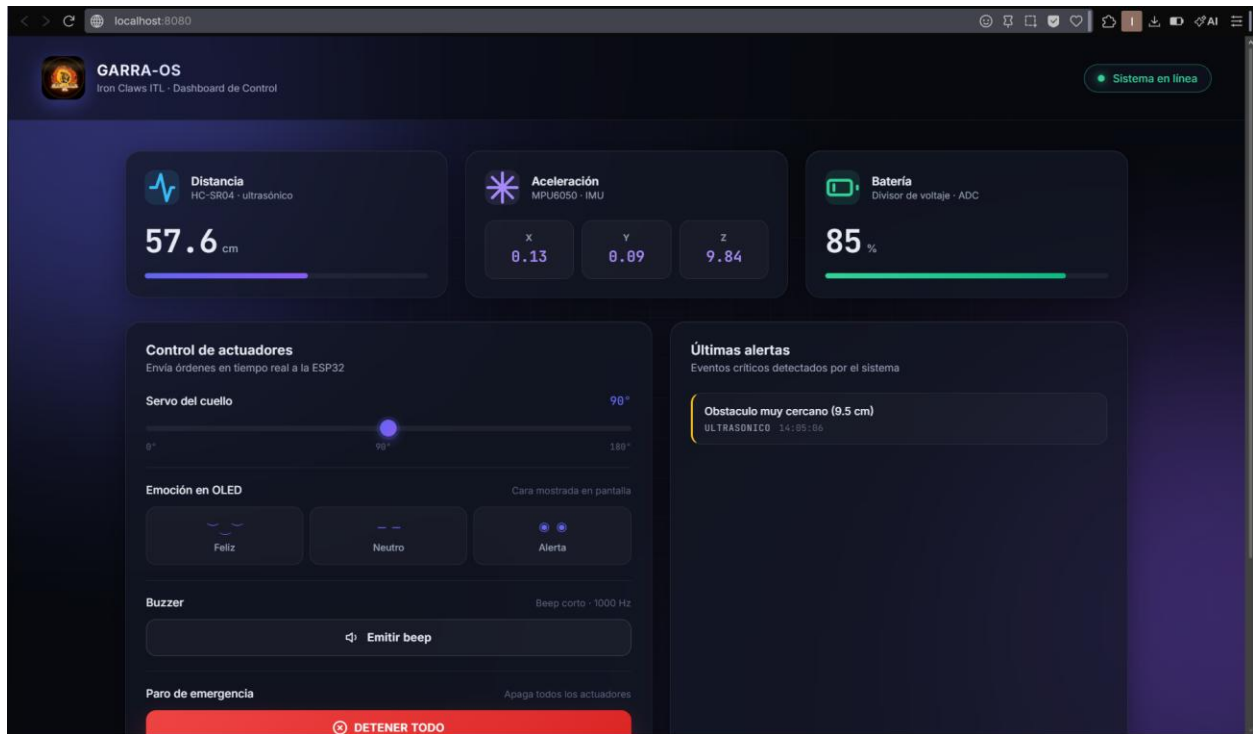
4.1 Monitoreo en tiempo real

- Indicador superior con el estado general del sistema (online/offline) basado en la rama /estado de Firebase y el mecanismo LWT (Last Will and Testament) de MQTT.
- Tres tarjetas con la última lectura de los sensores: distancia ultrasónica, aceleración en los ejes X/Y/Z y porcentaje de batería con barra de color (verde / ámbar / rojo).
- Lista lateral con las últimas 5 alertas, ordenadas de más reciente a más antigua, con código de color por severidad (alta / media / baja).

4.2 Control bidireccional

- Slider continuo de 0° a 180° para el servo del cuello.

- Tres botones para cambiar la emoción mostrada en la OLED del robot (feliz / neutro / alerta).
- Botón para emitir un beep del buzzer (1000 Hz, 300 ms).
- Botón de paro de emergencia que apaga todos los actuadores y queda visualmente diferenciado en color rojo, con confirmación previa.



5. Garantía de privacidad

De acuerdo con la rúbrica, queda prohibido el almacenamiento de rostros o imágenes identificables. El sistema cumple esto de la siguiente manera:

- Firebase únicamente almacena datos numéricos (distancia en cm, porcentaje de batería, aceleración en m/s^2) y de estado (online, severidad, valor de actuador).
- No hay ningún flujo de video ni captura de imágenes que llegue a la base de datos.
- Las reglas de seguridad de Firebase (archivo `firebase_rules.json`) exigen autenticación válida tanto para lectura como para escritura, e imponen validación de los valores aceptados en /comandos (rangos numéricos para el servo, lista cerrada de emociones para la OLED).
- Las credenciales de la cuenta de servicio del SDK admin se mantienen fuera del repositorio mediante `.gitignore`.

6. Análisis individual por integrante

Cada miembro del equipo expone a continuación un reto específico que enfrentó durante la integración con Firebase, la solución implementada y una conclusión personal sobre la experiencia.

Alcalá Ramos Luz Estefanía

Problema encontrado: Durante las primeras pruebas implementé la lectura de comandos desde Firebase usando polling cada 2 segundos. Esto generaba una latencia notoria (cerca de 3 segundos) entre el momento en que se tocaba un botón en el dashboard y la respuesta física del servo en la ESP32, además de que la cuota de lecturas de Firebase se consumía muy rápido.

Solución aplicada: Reemplacé el polling por el mecanismo stream del SDK admin de Python (`ref.listen()`), que mantiene una conexión SSE permanente con Firebase y dispara un callback en cuanto cambia el nodo escuchado. La latencia bajó a menos de 400 ms y el consumo de cuotas disminuyó drásticamente porque el canal queda abierto en vez de abrir una petición cada ciclo.

Conclusión personal: Entender la diferencia entre arquitecturas push y pull fue clave. En sistemas IoT donde se requiere reacción inmediata, suscribirse a eventos siempre va a ganarle al polling, tanto en latencia como en costo. Esta lección la voy a aplicar también en proyectos donde se use WebSocket o MQTT.

Bahena Mora Emilio Salvador

Problema encontrado: Al sincronizar la interfaz web con Firebase, los valores de los sensores parpadeaban de manera errática cada vez que llegaba una actualización. El problema era que reescribía el DOM completo cada vez que se disparaba `onValue`, lo que rompía animaciones y provocaba un efecto visual desagradable.

Solución aplicada: Cambié el patrón: en lugar de regenerar la tarjeta completa, solo modifiqué el `textContent` del elemento afectado y aplico una clase CSS `".flash"` que dispara una animación de medio segundo. Como la transición está en CSS y no en JS, el repintado es mucho más fluido. Además, agregué barras de progreso con `transition: width` que dan la sensación de continuidad entre lecturas.

Conclusión personal: El diseño web no es solo estética. Una interfaz que reacciona suavemente a los cambios transmite confianza en los datos. En sistemas industriales o de monitoreo, un parpadeo descontrolado puede hacer dudar al operador. Vale la pena invertir tiempo en CSS limpio y bien estructurado.

Casas Bastidas José Iván

Problema encontrado: En la primera versión, las reglas de seguridad de Firebase estaban abiertas (`.read` y `.write` en `true`) para poder hacer pruebas rápidas. Al cerrar la base, descubrí que el SDK admin de Python seguía funcionando, pero el navegador empezó a recibir errores `PERMISSION_DENIED` y el dashboard se rompió.

Solución aplicada: Configuré autenticación anónima en Firebase Authentication y modifiqué el front-end para invocar `signInAnonymously()` antes de cualquier operación de lectura/escritura. Actualicé las reglas para distinguir entre lecturas (`auth != null`) y escrituras críticas en `/telemetria`, `/eventos` y `/alertas` (solo accesibles para tokens con `claim admin == true`, que se aplica al service account de Python). El nodo `/comandos` sí es escribible por cualquier usuario autenticado, porque ahí es donde el dashboard manda órdenes.

Conclusión personal: Las reglas de seguridad son tan importantes como el código de la aplicación. Una base de datos abierta es una catástrofe esperando ocurrir, especialmente cuando se manejan actuadores físicos. Aprendí que separar permisos por nodo y por tipo de cliente (admin vs. usuario anónimo) es una práctica que se debe aplicar siempre.

Fischer González Patrick

Problema encontrado: Implementé el control del slider del servo y noté que cada milímetro que arrastraba el cursor disparaba una escritura en Firebase. Esto generaba decenas de eventos por segundo, saturaba el listener del servidor Python y el servo se movía a saltos porque le llegaban órdenes contradictorias en milisegundos.

Solución aplicada: Cambié el evento de `input` por `change` en el slider, de modo que solo se escribe en Firebase cuando el usuario suelta el control. El feedback visual sí se actualiza con `input` (para que el número del ángulo cambie en vivo), pero el comando solo se manda una vez al final. Para el botón de paro de emergencia añadí también un `confirm()` y un reset automático del flag `/comandos/paro` a `false` después de 1 segundo, para que pueda volver a usarse sin quedar enganchado.

Conclusión personal: El debouncing y throttling no son solo conceptos de optimización web: en IoT son indispensables porque cada escritura cuesta cuota de red y, sobre todo, puede generar un comportamiento físico errático del hardware. Pensar en el ciclo completo (UI → red → microcontrolador → motor) cambia la forma en que uno diseña la interfaz.

7. Checklist de entrega

- ✓ **Enlace al repositorio:** código del dashboard y del puente Python, ambos documentados con encabezado de integrantes y objetivo.
- ✓ **Firebase operativo:** datos reales entrando a /telemetria, /eventos y /alertas con timestamp del servidor.
- ✓ **Tres tipos de eventos con timestamp:** telemetria, alerta y actuador, marcados con {".sv": "timestamp"}.
- ✓ **Dashboard funcional:** estado general, lectura de los 3 sensores y log dinámico de las últimas 5 alertas.
- ✓ **Control remoto:** slider del servo, botones de emoción de la OLED, botón de buzzer y paro de emergencia.
- ✓ **Garantía de privacidad:** Firebase solo almacena datos numéricos y de estado, sin imágenes ni datos faciales.
- ✓ **Análisis individual:** Problema / Solución / Conclusión por cada integrante (apartado 6).
- ✓ **Enlace al repositorio de GITHUB:** <https://github.com/Ivan-Casas/GARRA-OS-Persistencia-en-la-Nube-y-Dashboard-de-Control-Firebase-Python-Interfaz-/tree/main>

8. Conclusiones generales

La integración de Firebase con el sistema GARRA-OS confirmó que separar capas (hardware, transporte MQTT, persistencia en la nube, interfaz) reduce notablemente la complejidad. Cada capa se puede depurar y reemplazar de forma aislada: el dashboard funciona sin saber MQTT, la ESP32 no necesita hablar HTTP y el puente Python tiene una responsabilidad única bien delimitada.

Los principales aprendizajes del equipo fueron:

- A. La importancia de usar mecanismos push como streams de Firebase y suscripciones MQTT en vez de polling,
- B. Que las reglas de seguridad de la base de datos son parte del diseño desde el día uno y no algo que se agrega al final.
- C. Que el control bidireccional de actuadores físicos exige tratar con cuidado la frecuencia de escritura para no generar comportamientos erráticos en el hardware.